به نام خالق زیبایی ها



گزارش پروژه پایانی درس هوش مصنوعی مقدماتی

نام استاد:

دکتر شهاب الدین نبوی

مشخصات دانشجو:

مارال اسمی خانی

مهندسی صنایع

زمستان 1403

1.در بخش اول تمامی کتابخانه هایی که در روند نوشتن کد لازم شد قرار داده شده است.

2.سپس فایلی که به عنوان dataset وجود داشت فراخوانی شده است.

3.با توجه به اینکه داده های موجود در data set از feature ها و label تشکیل شده بود label را به عنوان target در نظر گرفتم column ها را به دو دسته ی numericals و categoricals طبقه بندی کردم.

4.حال طبق تسک اول پروژه بررسی می کنیم که چه مقدار داده گم شده وجود دارد در اینجا از متد isnull استفاده می کنیم که مشاهده می شود در این دیتا ست داده ی گمشده(خالی ای) وجود ندارد.

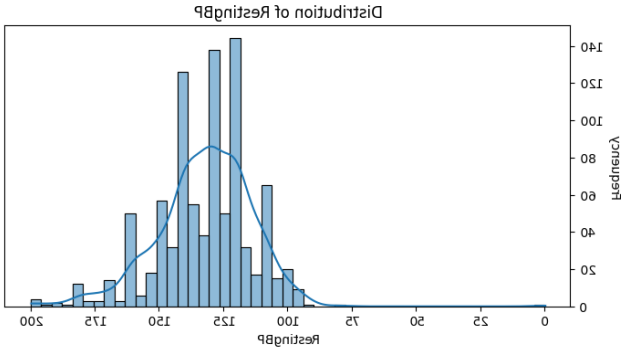
5.با این حال عملیات imputation را برای داده ها انجام دادم چون ممکن است داده های جدیدی به دیتا ست اولیه در آینده اضافه شود و آن داده ها در برخی ستون ها بدون مقدار باشند.

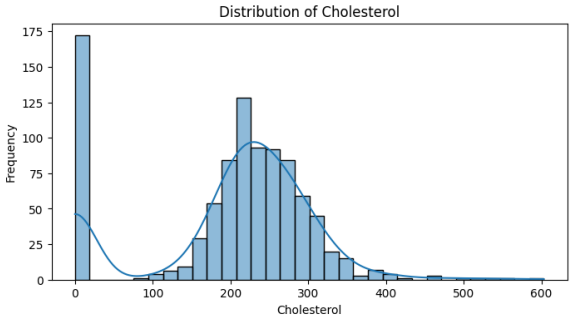
6.در مرحله ی بعد به سراغ ترسیم توزیع داده های numerical categorical , به صورت جداگانه رفتم و از متد های رسمشان استفاده کردم.

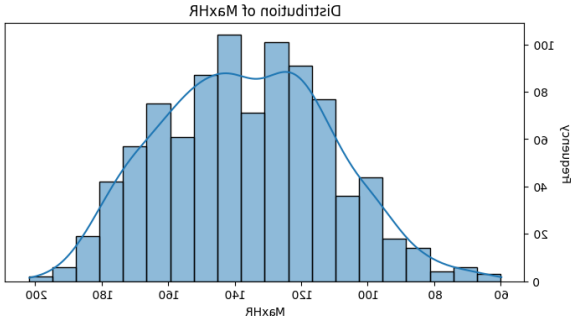
هر یک از این نمودار ها نشان می دهد که توزیع داده ها در مورد آن دسته ی خاص به چه شکل است مثلا در قسمت distribution of ChestPainType نشان داده شده است که تایپ ASY بیشترین تایپ در بین نمونه های موجود در dataset بوده است.

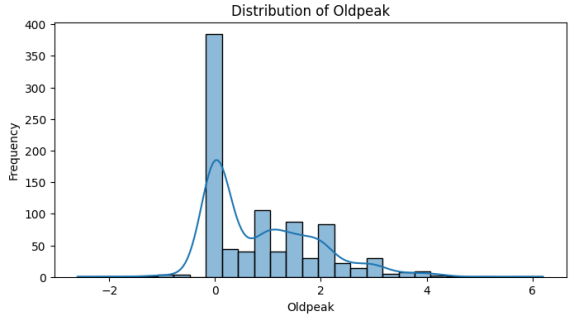
این کار به ما کمک می کند تا دید بهتری نسبت به داده ها و نحوه ی توزیع آن ها داشته باشیم.

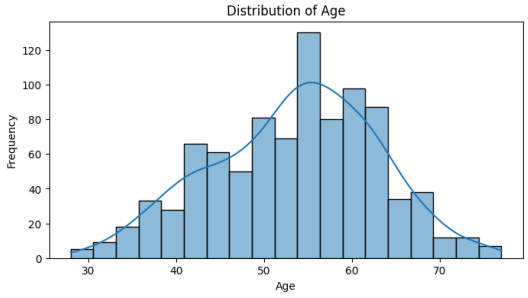
توزیع های به دست امده به شکل زیر است:

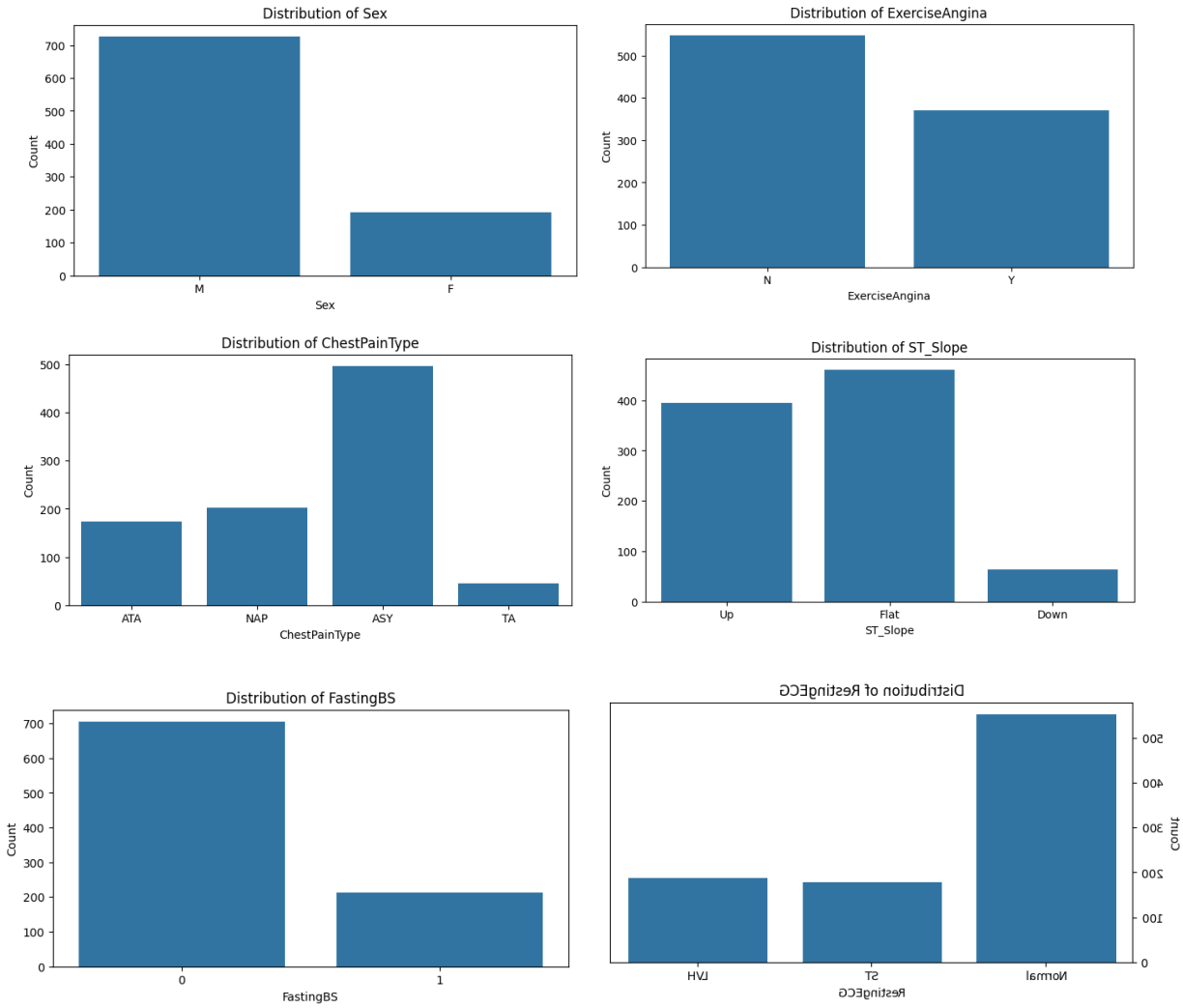












Kde مخفف Kernel Density Estimation است که یک روش آماری برای تخمین چگالی احتمال یک متغیر پیوسته است. وقتی از kde=True استفاده می‌کنیم، به برنامه می‌گوییم که به جای رسم هیستوگرام معمولی، از تخمین چگالی هسته‌ای (KDE) برای رسم نمودار استفاده کند.این روش، مخصوصا زمانی که می‌خواهیم توزیع داده‌ها را به‌طور پیوسته نمایش دهیم و از هیستوگرام‌های ناپیوسته پرهیز کنیم، مفید است.

7.در قسمت بعد وارد تسکی می شویم که لازم است داده های به صورت str را تبدیل به داده های عددی بکنیم .

برای انجام این کار لازم است برای داده های باینری از LabelEncoder و برای داده های رشته ای غیر باینری از OneHotEncoder استفاده کنیم.

از آنجایی که تعداد باینری ها دو عدد است و بقیه داده ها باید با onehotencoder تبدیل شوند برای هر دوی این مقادیر تابع های متفاوتی نوشتم که تابع apply_ohe متد onehotencoder را پیاده سازی می کند.

لازم به ذکر است که ابتدا تابع onehotencoder پیاده سازی شده ، ولی باید در آن تعریف کنیم که اگر داده ها به صورت باینری بودند آن ها را نادیده بگیر و از دستور drop="if_binory" استفاده کردیم.

sparse_output=False یعنی که می‌خواهیم نتیجه‌ی کدگذاری به‌صورت یک آرایه معمولی (dense array) یا همان آرایه کامل ارائه شود.

وقتی از متد onehotencoder استفاده می کنیم خروجی ای که به ما داده می شود از نوع array است که حاوی داده های رمزگذاری شده است .برای اینکه بتوانیم این داده ها را مشاهده و پردازش کنیم و همچنین بتوانیم در pandas دوباره از آن ها استفاده بکنیم لازم است که آن ها را به Dataframe تبدیل کنیم.

در کدی که برای label encoding نوشتیم ،نیازی به تبدیل داده‌ها به DataFrameجدید نداریم، چون در اینجا عملیات به‌طور مستقیم روی یک کپی از DataFrame اصلی (df_copy) انجام می‌شود که خودش یک pandas DataFrameاست.

بنابراین فقط لازم است تابع apply_le را برایش تعریف کنیم و از متد label encoderاستفاده کنیم.

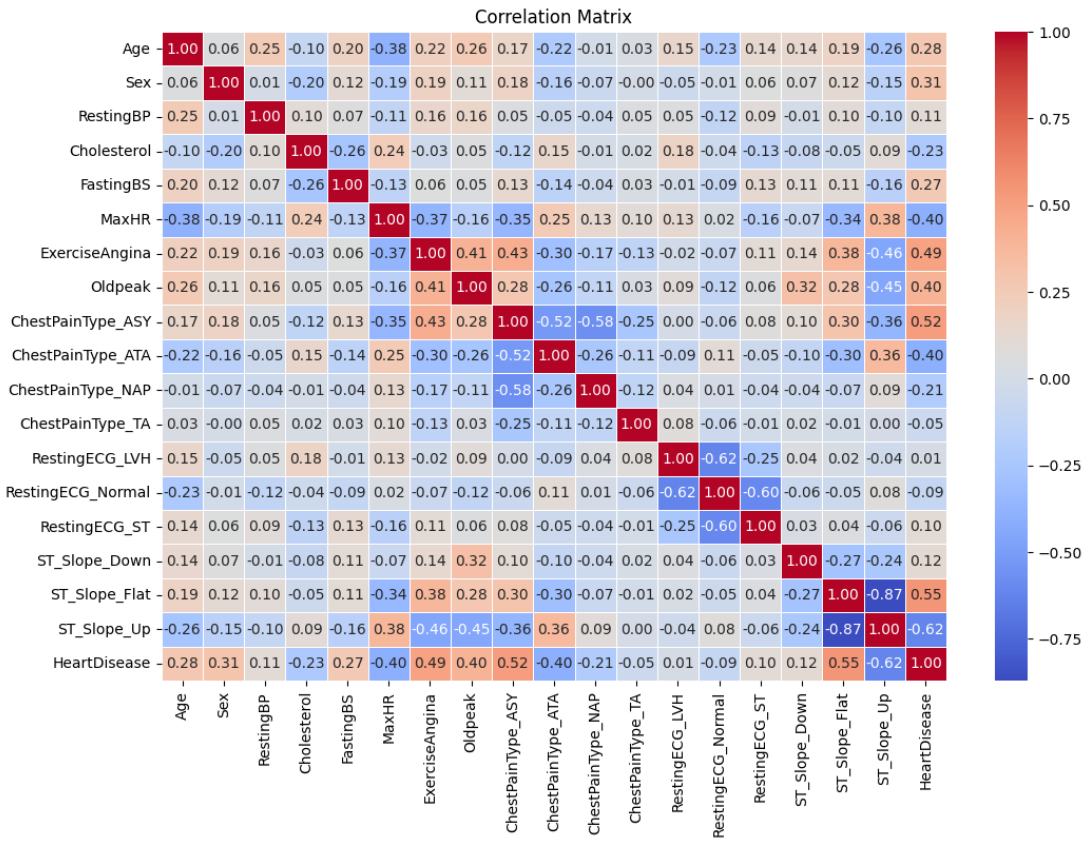
داده های targetهم که از ابتدا به صورت داده های عددی بودند پس نیازی به ایجاد تغییر در آن ها نیست.

حالا همه ی داده های ما به صورت عددی خواهد بود.

بعد از کشیدن ماتریس همبستگی در مرحله بعد متوجه شدم که داده های HeartDisease که همان targetما محسوب میشود در وسط ماتریس قرار گرفته بود پس آن را جابجا کرده و به سمت راست انتقال دادم تا اثر هر کدام از class ها روی target بهتر مشخص شود.

8.با انجام مرحله ی 7 قادر خواهیم بود که correlation بین داده ها را بررسی کنیم فقط کافیست که دیتا ست جدید را با متد correlation به شکل ماتریس همبستگی بکشیم.

همانطور که در ماتریس مشخص است تعدادی از feature ها با هم همبستگی دارند این یعنی اعداد به دست آمده نشان می دهد رابطه ای بین این feature ها وجود دارد و روی هم اثر گذار هستند.مهم ترین ستون احتمالا ستون سمت راست خواهد بود



که نشان می دهد تاثیر هر کلاس روی targetما به چه صورت است؟

9. در این بخش از پروژه، من تصمیم گرفتم که از مقیاس‌دهی استاندارد (Standard Scaling)برای داده‌های ویژگی‌ها استفاده کنم تا همه ویژگی‌ها در یک مقیاس یکسان قرار گیرند. این کار به مدل‌های یادگیری ماشین کمک می‌کند که عملکرد بهتری داشته باشند، زیرا بعضی از الگوریتم‌ها مانند رگرسیون خطی یا مدل‌های مبتنی بر فاصله) مثل (K-Nearest Neighbors به مقیاس داده‌ها حساس هستند.

ابتدا، من داده‌ها را به دو قسمت تقسیم کردم: ویژگی‌ها (X) و هدف .(y) در اینجا، ستون HeartDiseaseکه متغیر هدف (Target Variable)است از ویژگی‌ها جدا شد و به عنوان yدر نظر گرفته شد.

ویژگی‌ها در Xشامل همه ستون‌ها به جز HeartDisease هستند.

برای مقیاس‌دهی ویژگی‌ها، از کلاس StandardScalerاز کتابخانه sklearn.preprocessingاستفاده کردم.

StandardScalerبه این صورت عمل می‌کند که میانگین هر ویژگی را صفر و انحراف معیار آن را یک می‌کند. این کار باعث می‌شود که ویژگی‌ها در مقیاسی یکسان قرار گیرند و تأثیر هر ویژگی بر مدل به‌طور متوازن‌تر باشد.

پس از این که ویژگی‌ها و هدف از هم جدا شدند، من مقیاس‌دهی را روی ویژگی‌ها (X) انجام دادم. با استفاده از متد ()fit_transform، داده‌های Xرا به مقیاس استاندارد تبدیل کردم.

در نهایت، داده‌های مقیاس‌دهی شده که به صورت یک آرایه NumPy بازگشت داده شده بودند را به یک DataFrame تبدیل کردم تا بتوانم آن‌ها را به راحتی مدیریت و بررسی کنم. این کار همچنین نام ستون‌ها را حفظ کرد که برای تحلیل بعدی مفید است.

10.تقسیم داده‌ها به مجموعه‌های آموزش و تست:

در ابتدا، داده‌ها را به دو مجموعه‌ی آموزش و تست تقسیم کردم. این کار برای ارزیابی عملکرد مدل بر روی داده‌های جدید و ناشناخته انجام می‌شود. به این ترتیب:

مجموعه آموزشX_train و y_train شامل 80٪ از داده‌ها می‌شود.

مجموعه تستX_test و y_test شامل 20٪ از داده‌ها می‌شود.

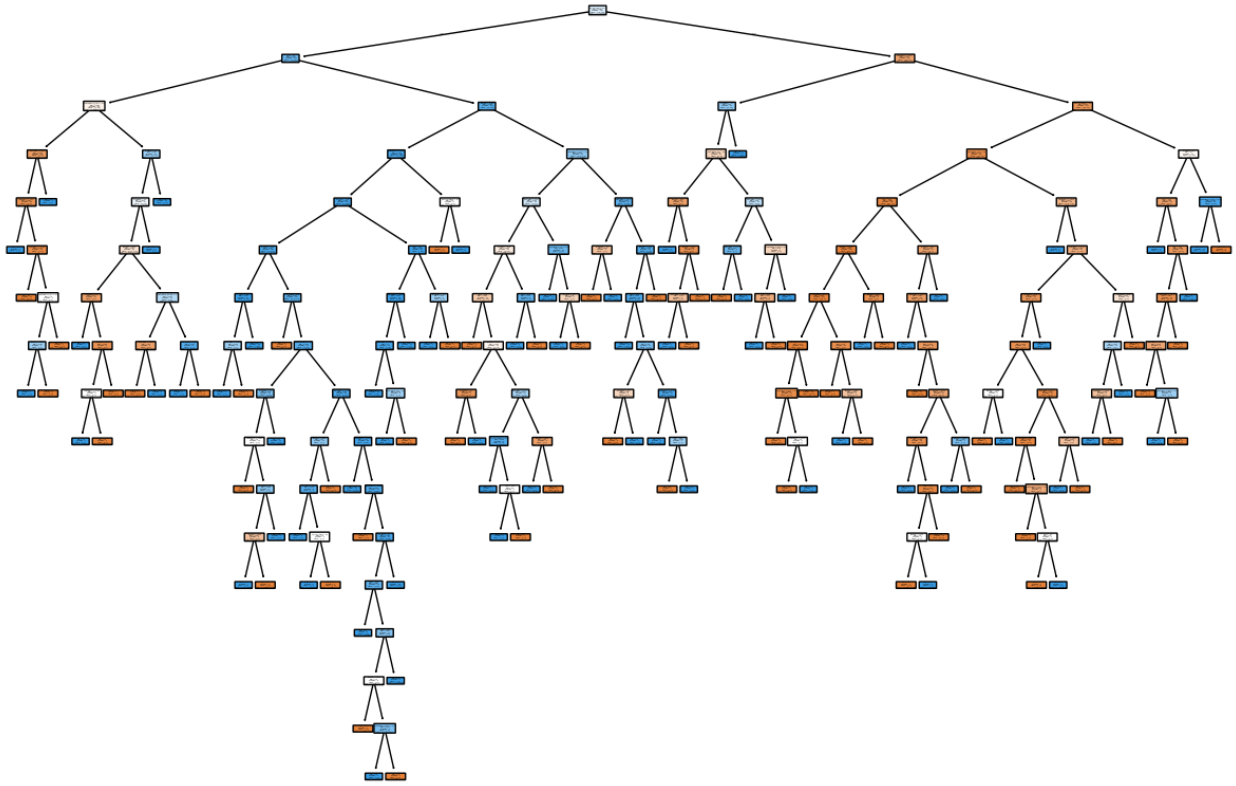
برای این تقسیم‌بندی از تابع train_test_split از کتابخانه sklearn.model_selection استفاده کردم. به علاوه، برای اطمینان از قابلیت تکرار نتایج، پارامتر random_state=42 برای تصادفی‌سازی داده‌ها انتخاب شد.

دلیل انتخاب 42 هم اینناست که استفاده از این عدد توسط برنامه نویسان برای این بخش متداول است.

در ادامه این بخش، از الگوریتم درخت تصمیم برای آموزش مدل استفاده کردم. این الگوریتم یکی از مدل‌های یادگیری نظارت‌شده است که به ویژه برای مسائل دسته‌بندی کاربرد دارد.

مدل درخت تصمیم با استفاده از DecisionTreeClassifierاز کتابخانه sklearn.treeساخته شد.

پارامتر random_state=42برای تنظیم تصادفی‌سازی درخت و اطمینان از تکرارپذیری نتایج انتخاب شد.



بعد از ساخت مدل، از متد fit()برای آموزش آن روی مجموعه‌ی آموزش (X_train, y_train) استفاده کردم.

متد fit()در یادگیری ماشین برای آموزش مدل و یادگیری پارامترهای آن از داده‌ها استفاده می‌شود، در حالی که fit_transform()ترکیبی از fit()و transform()است که همزمان مدل را آموزش داده و تغییرات لازم را روی داده‌ها اعمال می‌کند. از طرف دیگر، transpose()برای چرخاندن ابعاد یک ماتریس یا آرایه به کار می‌رود، به این صورت که سطرها را به ستون‌ها و ستون‌ها را به سطرها تبدیل می‌کند. بنابراین، fit()و fit_transform()مربوط به آموزش مدل و پردازش داده‌ها هستند، در حالی که transpose()مربوط به تغییر ساختار داده‌هاست.

بعد از آموزش مدل، پیش‌بینی‌هایی برای مجموعه تست (X_test) انجام دادم.

پیش‌بینی‌ها با استفاده از متد predict() به‌دست آمدند.

سپس دقت مدل با استفاده از تابع accuracy_score از کتابخانه sklearn.metrics محاسبه شد و نمایش داده شد.

برای ارزیابی دقیق‌تر عملکرد مدل، من از classification استفاده کردم. این گزارش شامل معیارهای زیر است:

Precision:درصد پیش‌بینی‌های درست برای هر کلاس.

Recall:درصد موارد صحیح شناسایی‌شده از کل موارد واقعی.

F1-Score:میانگین هماهنگ precision و recall که برای مدل‌های نابرابر (imbalanced) بسیار مفید است.

این گزارش با استفاده از تابع classification_report() از کتابخانه sklearn.metrics تولید شد.

:(Confusion Matrix)

به‌منظور ارزیابی دقیق‌تر عملکرد مدل، از ماتریس اشتباهات (confusion matrix)استفاده کردم. این ماتریس، تعداد پیش‌بینی‌های صحیح و غلط برای هر کلاس را نمایش می‌دهد و به تحلیل بیشتر عملکرد مدل کمک می‌کند.

این ماتریس با استفاده از تابع confusion_matrix()از کتابخانه sklearn.metricsبه‌دست آمد.

در نهایت، برای تجسم نحوه تصمیم‌گیری مدل درخت تصمیم، از تابع plot_tree() استفاده کردم. این تابع درخت تصمیم را به‌صورت گرافیکی نمایش می‌دهد، که در آن ویژگی‌ها و تصمیم‌گیری‌های مدل بر اساس گره‌ها نشان داده می‌شود.

پارامترهای filled=Trueبرای رنگ‌آمیزی گره‌ها و class_namesبرای نامگذاری کلاس‌ها در نظر گرفته شدند.

این تصویر به‌وسیله matplotlibو با تنظیم اندازه‌ی نمودار به ابعاد ۱۵x۱۰ نمایش داده شد.

11 .استخراج اهمیت ویژگی‌ها:

در ابتدا، با استفاده از ویژگی feature_importances_ در مدل درخت تصمیم، اهمیتی که مدل برای هر ویژگی در مجموعه آموزش اختصاص داده را استخراج کردم. این ویژگی یک آرایه است که هر مقدار در آن نشان‌دهندهٔ میزان تأثیر آن ویژگی در تصمیم‌گیری مدل است.

اهمیت هر ویژگی معمولاً در محدودهٔ [0, 1]قرار دارد و مجموع اهمیت‌های همهٔ ویژگی‌ها برابر با 1 می‌شود. ویژگی‌هایی که اهمیت بیشتری دارند، معمولاً نقشی کلیدی در تقسیم‌بندی داده‌ها ایفا می‌کنند.

سپس، این داده‌ها را در یک DataFrameقرار دادم که شامل دو ستون است:

Feature: نام ویژگی‌ها.

Importance: اهمیت اختصاص داده‌شده به هر ویژگی توسط مدل درخت تصمیم.

این DataFrame به من این امکان را داد تا بتوانم داده‌ها را به راحتی پردازش کنم و آن‌ها را نمایش دهم.

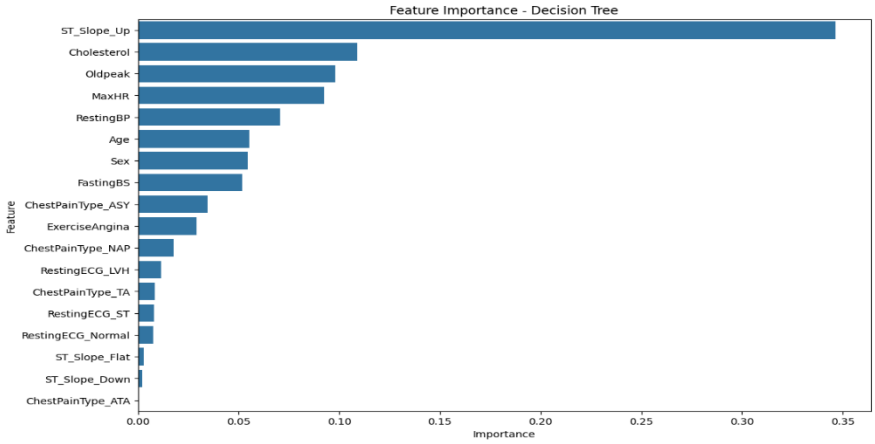
برای این که ویژگی‌هایی که بیشترین تأثیر را دارند در بالای جدول قرار بگیرند، DataFrameرا بر اساس مقدار اهمیت مرتب کردم. این کار اجازه داد تا بتوانم ویژگی‌هایی که بیشترین تأثیر را دارند شناسایی کنم.

سپس، از 10ویژگی برتر از نظر اهمیت مدل برای پیش‌بینی استفاده کردم. این ویژگی‌ها مهم‌ترین پارامترهایی هستند که درخت تصمیم از آن‌ها برای تقسیم داده‌ها استفاده کرده است:.

برای نمایش بهتر اهمیت ویژگی‌ها و مشاهده مقایسه‌ای بین آن‌ها، از نمودار میله‌ای استفاده کردم. این نمودار، اهمیت هر ویژگی را به صورت بصری نمایش می‌دهد، و با استفاده از seaborn، نمودار را در ابعادی مناسب و خوانا ترسیم کردم.

محور افقی (x) نشان‌دهنده اهمیت ویژگی‌ها است.

محور عمودی (y) نشان‌دهنده نام ویژگی‌ها است.



12.کاهش ابعاد با PCA:

در این بخش، از تحلیل مولفه‌های اصلی Principal Component Analysis)یا (PCA برای کاهش ابعاد داده‌ها استفاده کردم .PCAیک تکنیک محبوب برای کاهش ابعاد است که ویژگی‌های اصلی داده‌ها را در ابعاد کمتری نگهداری می‌کند، در حالی که بیشترین واریانس موجود در داده‌ها حفظ می‌شود.

پارامتر n_components=2 به این معنی است که داده‌ها به دو مولفه اصلی PC1 و PC2 فشرده شدند.

داده‌های ورودی که مقیاس‌بندی شده بودند(X_scaled) ، به دو بعد جدید تبدیل شدند.

پس از انجامPCA ، داده‌های فشرده شده را در یک DataFrame به نام pca_df قرار دادم. این DataFrame شامل:

دو ستون PC1و PC2است که نشان‌دهنده مولفه‌های اصلی اول و دوم هستند.

یک ستون دیگر به نام HeartDiseaseاضافه کردم که هدف (وجود یا عدم وجود بیماری قلبی) را نشان می‌دهد تا ارتباط داده‌های فشرده شده با متغیر هدف قابل مشاهده باشد.

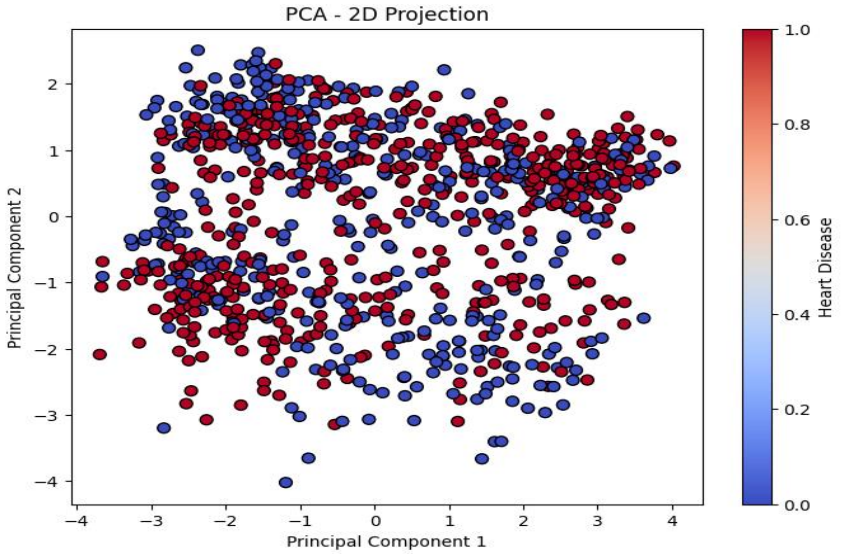
برای مشاهده بصری نتایجPCA ، از نمودار پراکندگی (scatter plot)استفاده کردم.

محور افقی نشان‌دهنده PC1و محور عمودی نشان‌دهنده PC2است.

نقاط بر اساس وجود یا عدم وجود بیماری قلبی رنگ‌بندی شده‌اند. رنگ‌ها از cmap='coolwarm'گرفته شده‌اند که رنگ‌ها را از آبی (برای عدم وجود بیماری) به قرمز (برای وجود بیماری) تغییر می‌دهد.

علاوه بر این، از colorbarبرای نمایش رنگ‌ها و ارتباط آن‌ها با بیماری قلبی استفاده کردم.

در انتها، نسبت واریانس توضیح داده‌شده توسط هر مولفه اصلی PC1 و PC2 را چاپ کردیم. این مقادیر نشان می‌دهند که هر مولفه چقدر از کل واریانس داده‌ها را توضیح می‌دهد. این اطلاعات به ما کمک می‌کند تا درک کنیم که هر مولفه چقدر اطلاعات اصلی داده‌ها را نگهداری کرده است.



13. خوشه بندی با K-Means :

برای بررسی تأثیر تعداد خوشه‌ها، من از یک بازه مختلف از تعداد خوشه‌ها (2تا 5خوشه) استفاده کردم.

برای هر تعداد خوشه، مدل KMeansاجرا شد و خوشه‌های مختلف داده‌ها شناسایی گردید. سپس برچسب‌های خوشه‌ها به داده‌ها اضافه شد و تعداد نمونه‌ها در هر خوشه نمایش داده شد. این اطلاعات به من این امکان را داد که ببینم داده‌ها چگونه بین خوشه‌های مختلف تقسیم‌بندی می‌شوند.

برای مثال:

با استفاده از pd.Series(kmeans.labels_).value_counts(), تعداد نمونه‌ها در هر خوشه چاپ می‌شود. این نتایج نشان می‌دهند که هر خوشه شامل چه تعداد داده است.

کاهش ابعاد با:PCA

پس از انجام خوشه‌بندی، برای تحلیل بهتر و تجسم داده‌ها، از PCA (Principal Component Analysis)برای کاهش ابعاد به سه بعد استفاده کردم. این کار به من کمک کرد که خوشه‌ها را در فضای سه‌بعدی ببینم و آن‌ها را تجزیه و تحلیل کنم.

PCAبا ۳ مولفه اصلی اعمال شد (n_components=3) تا بتوانم داده‌ها را به‌طور مؤثر در فضای سه‌بعدی نمایش دهم. این کمک کرد که تفاوت‌های بین خوشه‌ها و نحوه توزیع آن‌ها در فضای داده‌ها به‌طور بصری قابل مشاهده باشد.

نمایش نتایج به صورت سه‌بعدی:

برای هر تعداد خوشه، از نمودار سه‌بعدی برای تجسم نتایج PCA استفاده کردم. در این نمودار:

محور X، Yو Zنمایانگر سه مولفه اصلی PCA هستند که نشان‌دهنده‌ی ابعاد کاهش یافته داده‌ها هستند.

هر نقطه در این نمودار نشان‌دهنده یک نمونه است که با رنگ‌ها به خوشه‌های مختلف تعلق دارد. رنگ‌ها به‌طور خودکار توسط پالت viridisانتخاب می‌شوند.

برچسب‌ها برای خوشه‌ها به کمک رنگ‌ها در این نمودار مشخص شده‌اند.

با استفاده از fig.colorbar(scatter, label='Cluster')، یک نوار رنگی اضافه شده که نشان‌دهنده خوشه‌های مختلف است.

نسبت واریانس توضیح داده‌شده توسط PCA:

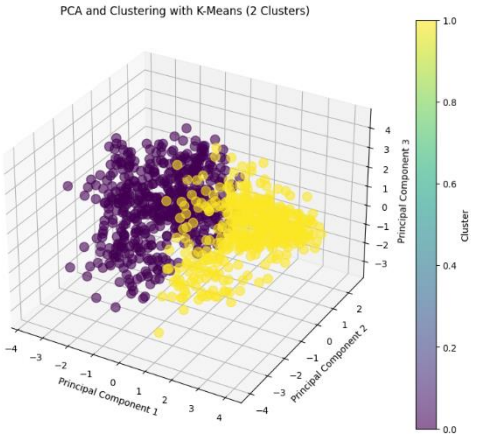
بعد از اعمالPCA ، من نسبت واریانس توضیح داده‌شده توسط هر مولفه اصلی را برای هر تعداد خوشه چاپ کردم. این اطلاعات به من کمک کرد که بفهمم هر مولفه چه مقدار از واریانس داده‌ها را توضیح می‌دهد و چقدر اطلاعات اصلی حفظ می‌شود.

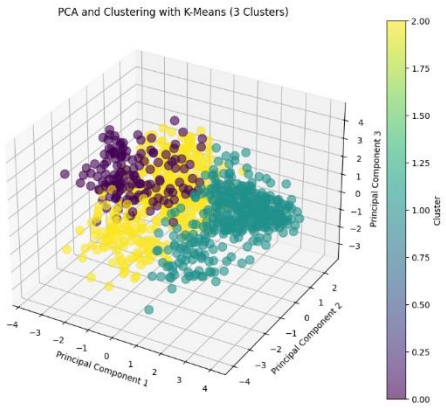
این نتایج با استفاده از pca.explained_variance_ratio_به‌دست آمد.

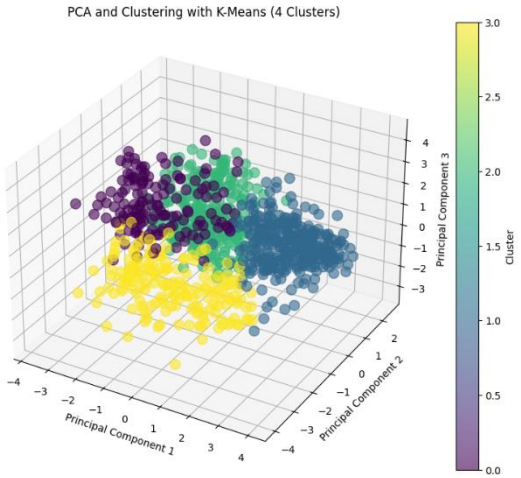
نتایج:

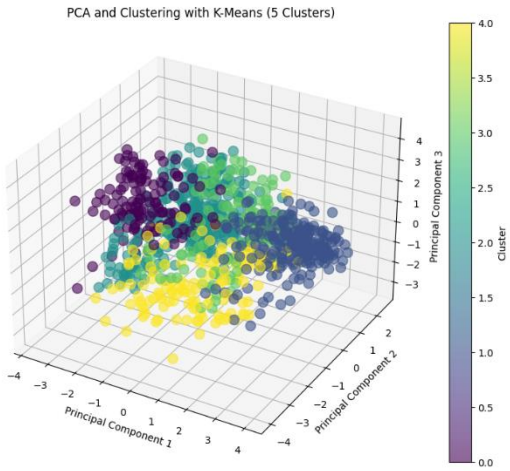
برای هر تعداد خوشه، تعدادی نمونه در هر خوشه، نسبت واریانس توضیح داده‌شده، و نمودارهای سه‌بعدی برای نمایش خوشه‌ها ارائه شد.

تجزیه و تحلیل نتایج و مقایسه‌ی نمودارهای سه‌بعدی به من کمک کرد تا بفهمم که با افزایش تعداد خوشه‌ها، خوشه‌بندی چگونه تغییر می‌کند و آیا تقسیم‌بندی داده‌ها بهتر می‌شود یا خیر.









14. . . 1. درخت تصمیم (Decision Tree) :

ابتدا مدل درخت تصمیم با استفاده از کلاس DecisionTreeClassifier از کتابخانه sklearn.treeساخته شد. برای جلوگیری از تصادفی بودن نتایج، از پارامتر random_state=42استفاده گردید.

پس از آموزش مدل روی داده‌های آموزشی(X_train, y_train) ، پیش‌بینی‌ها روی داده‌های آزمایشی (X_test) انجام و دقت مدل محاسبه شد.

علاوه بر دقت، گزارش طبقه‌بندی برای مدل درخت تصمیم چاپ شد که شامل معیارهایی مانند دقت(Precision) ، یادآوری (Recall)، و F1-Score است.

:Regression

مدل رگرسیون لجستیک با استفاده از کلاس LogisticRegression از sklearn.linear_model ایجاد و آموزش داده شد. مشابه درخت تصمیم، از random_state=42برای ثبات نتایج استفاده شد.

پیش‌بینی‌های مدل روی داده‌های آزمایشی انجام شد و دقت مدل محاسبه گردید.

گزارش طبقه‌بندی این مدل نیز نمایش داده شد تا ارزیابی بهتری از عملکرد آن در اختیار قرار گیرد

.

:XGBoost

مدل XGBoost(یک الگوریتم پیشرفته برای طبقه‌بندی و رگرسیون) از کتابخانه xgboostاستفاده شد.

مشابه مدل‌های قبلی، این مدل نیز روی داده‌های آموزشی آموزش دید و پیش‌بینی‌ها روی داده‌های آزمایشی انجام شد.

دقت مدل محاسبه و گزارش طبقه‌بندی آن چاپ شد.

:(SVM)

مدل SVMاز sklearn.svmبرای طبقه‌بندی استفاده شد. این مدل نیز با استفاده از random_state=42ایجاد و آموزش داده شد.

پیش‌بینی‌ها و دقت مدل محاسبه گردید و گزارش طبقه‌بندی مربوط به SVM نیز نمایش داده شد.

در انتها، دقت مدل‌ها چاپ شد که نشان‌دهنده درصد پیش‌بینی‌های صحیح هر مدل نسبت به کل داده‌های آزمایشی است.

همچنین، برای هر مدل، گزارش طبقه‌بندی ارائه شد که شامل معیارهایی مانند:

:Precisionدقت پیش‌بینی‌های مثبت.

:Recallتوانایی مدل در شناسایی تمامی موارد مثبت.

:F1-Scoreمیانگین هماهنگ precision و recall که برای مسائل با کلاس‌های نابرابر مفید است.

این بخش به مقایسه دقت و عملکرد کلی مدل‌های مختلف پرداختیم است. این مقایسه می‌تواند کمک کند تا بهترین مدل برای داده‌های موجود انتخاب شود.

همچنین، مدل‌ها از نظر دقت و گزارش طبقه‌بندی مقایسه شدند تا مشخص شود کدام مدل بهترین عملکرد را در پیش‌بینی بیماری قلبی دارد.

:15

ابتدا مدل‌ها را در یک دیکشنری قرار دادم تا از آن‌ها به راحتی استفاده کنم. برای هر مدل، ابتدا آموزش روی داده‌های آموزشی (X_train, y_train)انجام شد و سپس مدل‌ها بر روی داده‌های آزمایشی (X_test) پیش‌بینی کردند. به‌منظور ارزیابی دقیق عملکرد، چندین معیار مختلف برای هر مدل محاسبه شد.

معیارهای ارزیابی:

برای ارزیابی عملکرد هر مدل، من از معیارهای زیر استفاده کردم:

دقت :(Accuracy) درصد پیش‌بینی‌های صحیح نسبت به کل داده‌های آزمایشی.

دقت پیش‌بینی :(Precision) توانایی مدل در شناسایی نمونه‌های مثبت صحیح.

یادآوری :(Recall) توانایی مدل در شناسایی تمامی نمونه‌های مثبت.

:F1-Scoreمیانگین هارمونیک بین دقت پیش‌بینی و یادآوری که برای مدل‌هایی با داده‌های نابرابر بسیار مفید است.

:AUC (Area Under the Curve)مساحت زیر منحنی ROC که میزان توانایی مدل در تفکیک کلاس‌های مثبت و منفی را نشان می‌دهد.

زمان اجرا :(Execution Time) مدت زمانی که برای آموزش مدل و انجام پیش‌بینی‌ها صرف شد.

ارزیابی مدل ها:

برای هر مدل، ابتدا زمان اجرای آموزش و پیش‌بینی ثبت شد. پس از آن، پیش‌بینی‌ها انجام شد و دقت، دقت پیش‌بینی، یادآوری، F1-Scoreو AUC محاسبه گردید.

برای ارزیابی AUC، منحنی ROC ترسیم شد و مساحت زیر منحنی محاسبه شد. این معیار نشان می‌دهد که مدل چقدر قادر است کلاس‌های مختلف (مثلاً وجود یا عدم وجود بیماری قلبی) را تفکیک کند.

برای هر مدل، گزارش طبقه‌بندی (Classification Report)چاپ شد که شامل دقت پیش‌بینی، یادآوری و F1-Score برای هر کلاس است.

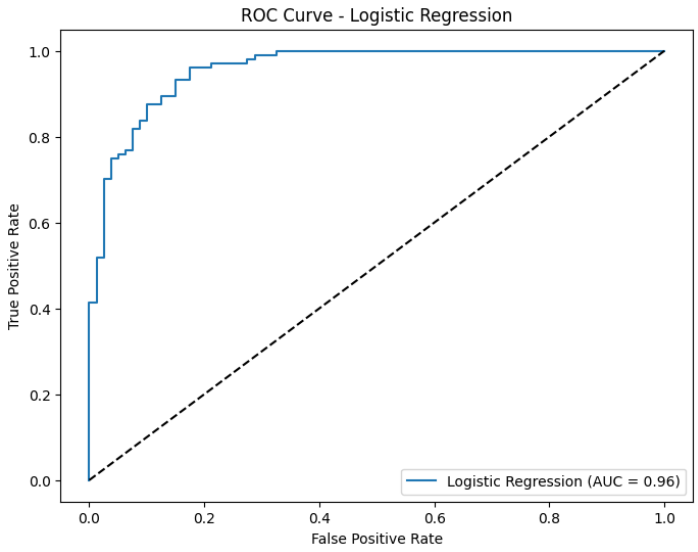
نمودار های ROC:

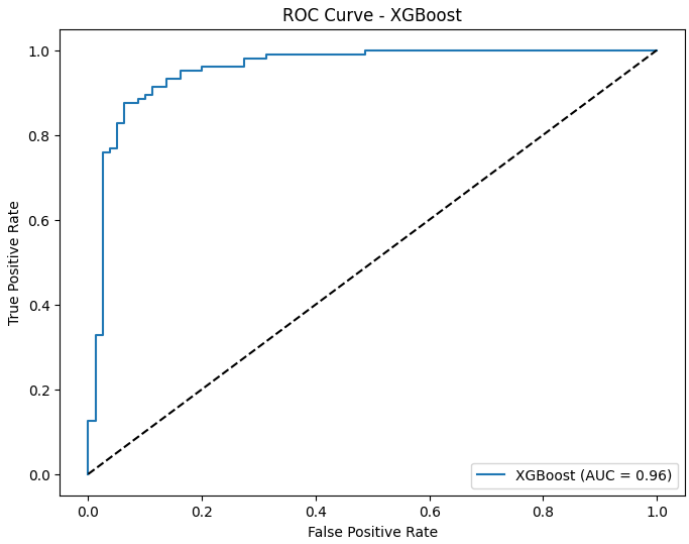
به‌منظور ارزیابی بصری هر مدل، نمودار ROC Curve برای هر مدل ترسیم شد. در این نمودار:

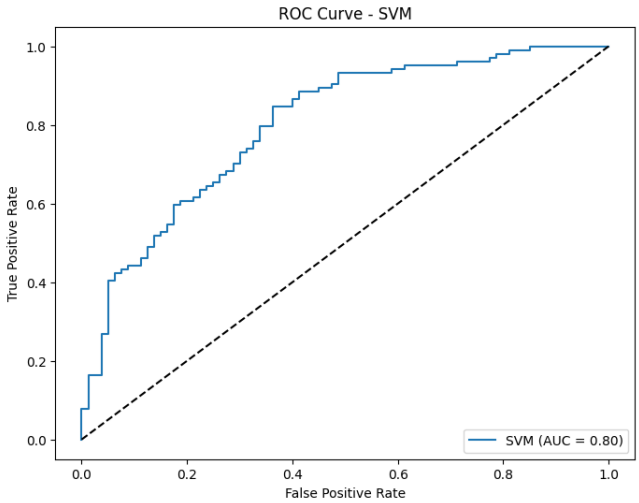
محور افقی False Positive Rate (FPR)و محور عمودی True Positive Rate (TPR)است.

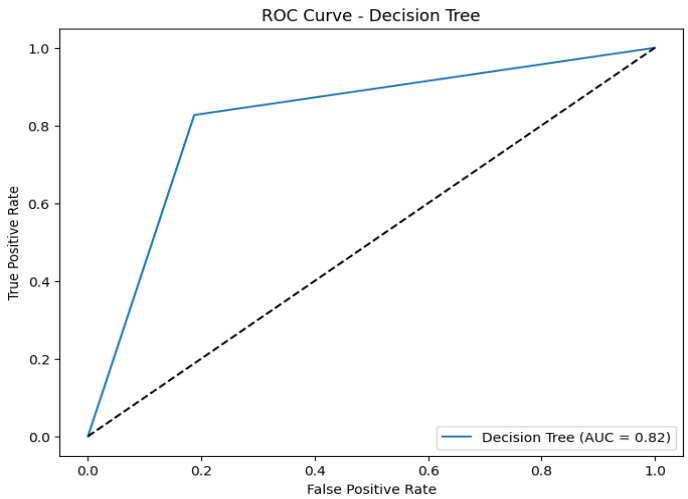
خط مورب در این نمودار نشان‌دهنده پیش‌بینی‌های تصادفی است.

با استفاده از AUC، می‌توان مدل‌ها را از نظر توانایی تفکیک کلاس‌ها مقایسه کرد.









مقایسه مدل ها:

پس از ارزیابی هر مدل، نتایج به یک DataFrame ذخیره شد و مدل‌ها بر اساس AUCمرتب شدند. این کار به من این امکان را داد که بهترین مدل را از نظر توانایی تفکیک کلاس‌ها شناسایی کنم.

در انتها، بهترین مدل بر اساس AUC شناسایی و چاپ شد. که این مدل Logistic Regression در نظر گرفته شد.